SAMSUNG INNOVATION CAMPUS · SIC 8 · BIG DATA ENGINEERING TRACK

PROJECT 03 · SUPPLY CHAIN & INTELLIGENT LOGISTICS

Enterprise Fleet Telematics Lakehouse & Dynamic Logistics Warehouse

Full project plan: architecture, data strategy, model, KPIs, tooling, team split and 6-day execution schedule

Delivery model	Hybrid: local Docker platform + AWS cloud pathway running the same code
Team	3 engineers: Member A (Data & Streaming), Member B (Processing & Lakehouse), Member C (Platform, Cloud & Analytics)
Duration	6 build days (Day 1–6), demo after Day 6
Inputs	Kaggle "Logistics Operations Database" (DS1) and "Logistics and Supply Chain Dataset" (DS2), plus a custom 5,000-vehicle telemetry generator
Document	Plan v1.0 · 22 Oct 2026 · prepared under senior data-engineering review

1 Executive summary

We will build a working telematics data platform for a simulated fleet of **5,000 refrigerated and dry delivery vehicles** moving on fixed logistics corridors. Real Kaggle data supplies the master data (trucks, drivers, routes, facilities, loads, fuel history) and calibrates realistic behaviour; a Python generator produces live telemetry that references those same IDs. Events flow through Kafka (local) and Kinesis (AWS) into an immutable bronze zone, are cleaned and enriched by PySpark, and land in Apache Iceberg silver and gold tables with a star schema. Trino and Athena serve three business KPIs to Superset dashboards, and Airflow orchestrates the whole path with quality gates.

Stakeholders & use case

Fleet operations manager (fuel cost per route), cold-chain quality lead (temperature compliance), safety coach (driver idling and harsh driving). They need one consistent, queryable view instead of three disconnected streams.

Why hybrid

The handbook treats local and cloud as equally valid. We build once and run twice: `ENV=local` on Docker and `ENV=aws` on S3/EMR/Athena. Local is the main demo; AWS proves cloud readiness.

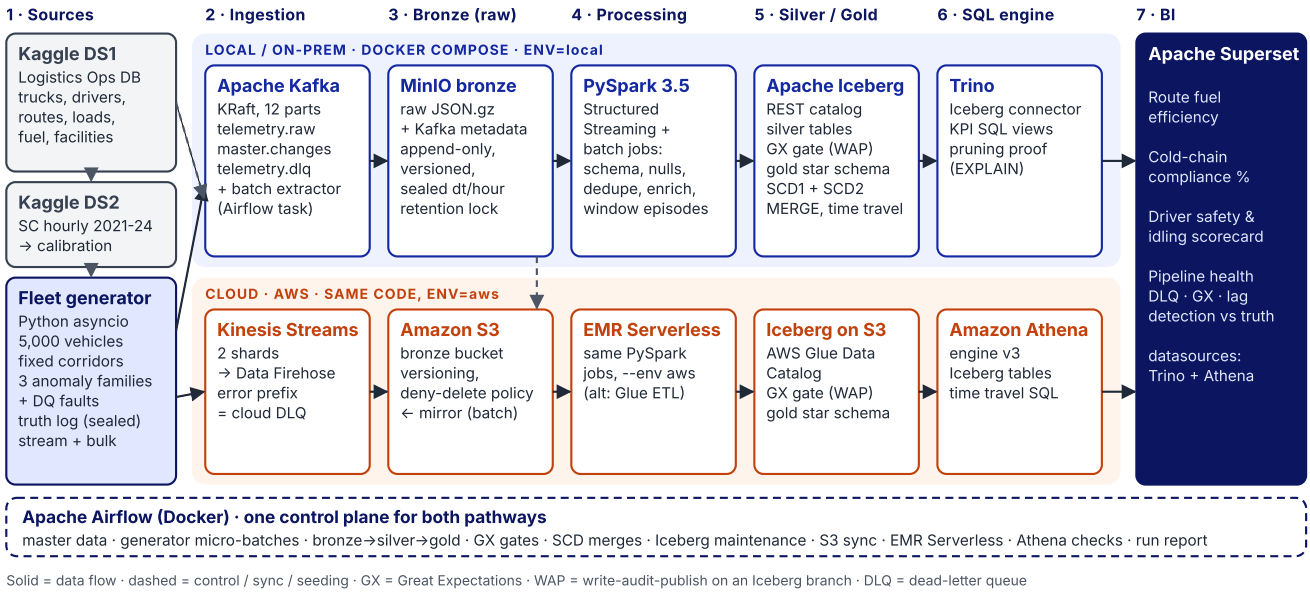
What makes it gradeable

Every anomaly the generator injects is written to a sealed truth log. We can then show that detection rules find what was injected (precision and recall), without the rules ever reading the labels.

Measurable objectives (what "done" means)

#	Objective	Target	Evidence
O1	Simulate the required fleet on fixed corridors	5,000 distinct `vehicle_id` in silver for one run	Trino count query + generator config
O2	All telemetry fields present	GPS lat/lon, OBD-II fuel rate, RPM, engine temp, reefer temp: ≥ 99% non-null outside injected gaps	GX silver suite report
O3	Nothing silently lost	bronze count = silver + quarantine + DLQ (exact)	Reconciliation query, ingestion log
O4	No duplicates in curated data	0 duplicate (vehicle_id, event_ts, seq_no) in silver	GX uniqueness check
O5	Anomaly detection matches injected truth	Recall ≥ 95%, precision ≥ 90% per family	Detection-vs-truth report
O6	KPIs are trustworthy	Dashboard = SQL = hand calculation on a sampled vehicle (±0.1%)	KPI reconciliation sheet
O7	Freshness	Stream → silver ≤ 2 min; event → gold KPI ≤ 15 min	Freshness panel on health dashboard
O8	Reproducible	Fresh clone → `make up && make demo` on a second laptop in ≤ 30 min	Recorded run + README
O9	Measured, not assumed, performance	Report achieved events/s, job runtimes, RAM/CPU per run	Benchmark CSV + Spark UI screenshots

2 Solution architecture



End-to-end data flow

1. **Seed:** Airflow downloads DS1 and DS2, writes them unchanged to bronze with a checksum manifest, and loads the dimensions. DS2 is profiled once to produce `config/calibration.json` (hour-of-day traffic factors, fuel-rate and temperature distributions, driver-behaviour profiles).
2. **Generate:** the generator expands DS1 trucks and drivers to 5,000 vehicles and assigns them DS1 routes and loads. It emits a ping per vehicle at a configurable interval in two modes: *stream* (Kafka + Kinesis) and *bulk micro-batch* (hourly files picked up by Airflow).
3. **Land:** a Spark Structured Streaming job writes every Kafka record, as received, to `s3a://bronze/telemetry/dt=/hour=/` with topic, partition, offset and ingest time. In AWS, Firehose does the same into S3. Bronze is append-only and is never rewritten.
4. **Clean (silver):** PySpark validates against the data contract, routes failures to DLQ, handles nulls, deduplicates, enriches with dimensions (broadcast joins) and computes per-ping features with window functions.
5. **Model (gold):** MERGE into SCD dimensions; build transaction, episode and snapshot facts; compute KPI marts. Every write passes a Great Expectations gate on an Iceberg audit branch before it is published to `main`.
6. **Serve:** Trino (local) and Athena (AWS) expose gold tables and KPI views; Superset reads both.
7. **Operate:** Airflow runs the chain, retries with backoff, and writes a run report (counts, GX results, timings) to `ops.pipeline_runs`.

> **One codebase, two targets.** Each job reads `conf/{local,aws}.yaml` for storage URIs, catalog type (Iceberg REST vs Glue) and broker (Kafka vs Kinesis). No logic is duplicated between environments, so the same tests prove both pathways.

3 Data strategy: how the Kaggle data and the generator fit together

> **Day 1 verification required.** The file and column lists below are what we expect from the two Kaggle pages. Member A confirms the real schemas in the Day 1 profiling report, and the data contract is adjusted before generator work starts. Anything renamed is handled in a small adapter layer, not in downstream jobs.

Source	Expected content	Role in our platform	Lands in
DS1 Logistics Operations Database (trucking ops 2022–2024)	trucks, trailers, drivers, customers, facilities, routes, loads, trips, fuel purchases, maintenance records, delivery events, safety incidents	**Master and reference data.** Seeds vehicle/driver profiles, corridors (origin–destination pairs), cargo types and facility geofences. Fuel purchases give a real benchmark to sanity-check simulated km/L.	bronze/kaggle_ds1 → dim_vehicle, dim_driver, dim_route, dim_facility, dim_customer, dim_cargo, fact_fuel_purchase
DS2 Logistics & Supply Chain dataset (hourly, 2021–2024)	timestamped GPS, fuel consumption rate, IoT temperature, traffic congestion, driver behaviour/fatigue scores, delay and risk indicators	**Calibration and history.** Fits distributions the generator samples from, so simulated behaviour follows real patterns. Also kept as a historical context fact.	bronze/kaggle_ds2 → calibration.json; fact_network_conditions_hourly
Generator (ours)	live pings for 5,000 vehicles + master-data change events + sealed anomaly truth log	**The streaming workload** required by Pillar 1 and Project 03.	Kafka/Kinesis → bronze/telemetry → silver → gold

Linking the generator to the real data

- **Fleet expansion to 5,000:** vehicle `VH-00001…VH-05000` maps deterministically to DS1 truck `i mod N` and inherits make, model year, fuel profile and reefer capability. `source_truck_id` is kept for lineage, so every simulated vehicle traces back to a real record. Drivers are expanded the same way.
- **Fixed corridors:** each distinct DS1 origin–destination route becomes a corridor. City coordinates come from a free city lookup (e.g. SimpleMaps basic CSV); a polyline is built by interpolating waypoints every ~1 km. This is not road-accurate, and that is a documented limitation (OSRM road geometry is an optional enhancement).
- **Trips:** DS1 loads define cargo type (cold or dry), reefer setpoint, customer and destination facility. The generator replays loads as trips along their corridor.
- **Realism from DS2:** speed follows hour-of-day congestion factors, fuel rate follows the fitted distribution scaled by speed and load, reefer noise follows the IoT temperature variance, and drivers get a behaviour profile (calm / average / aggressive) from DS2 score quantiles.
- **Units:** if DS1 is imperial (miles, gallons), it is converted at the silver boundary to km and litres, with factors stored in `conf/units.yaml`. Gold is metric only.

Telemetry event contract (JSON, schema v1, validated with JSON Schema)

Field	Type / unit	Notes
`event_id`	UUID	Unique per emission; resends keep the same id.
`vehicle_id, driver_id, trip_id, route_id`	string	Foreign keys to DS1-derived masters.
`seq_no`	long	Monotonic per vehicle; with `event_ts` forms the dedupe key.
`event_ts` / `sent_ts`	ISO-8601 UTC	Simulated event time vs wall-clock send time (kept separate on purpose).
`lat, lon`	deg (WGS84)	GPS position.
`speed_kmh, max_decel_mps2`	km/h, m/s²	Peak deceleration since previous ping (for harsh-braking events).
`rpm, engine_temp_c, fuel_rate_lph, fuel_level_pct, odometer_km`	OBD-II	Engine and fuel signals.
`ignition_on, reefer_on`	bool	Needed for idling and cold-chain logic.

Field	Type / unit	Notes
`reefer_temp_c`, `reefer_setpoint_c`	°C	Null when no reefer; null burst = sensor gap.
`schema_version`, `source`	string	`v1` / `v2`; `stream` or `bulk`.

Other topics: `fleet.master.changes` (driver reassignment, depot moves, reefer swaps, route corrections, used to demonstrate SCD) and `fleet.telemetry.dlq`. A v2 schema adds `tire_pressure_kpa` on Day 4 to demonstrate Iceberg schema evolution.

Injected anomalies vs independent detection rules

Family	How the generator injects it (written to sealed truth log)	How the pipeline detects it (never reads labels)
Cold-chain breach	Reefer drift or door-open event: temperature rises 4–10 °C above setpoint for 15–90 min on cold loads. Rate configurable per vehicle-day.	Cold load and `reefer_temp_c > setpoint + 3 °C` for ≥ 15 consecutive minutes → one excursion episode (gaps-and-islands window).
Excessive idling	Vehicle stops away from facilities with engine running (RPM 550–900) for 12–60 min.	`ignition_on`, speed < 2 km/h, RPM 500–1000 for ≥ 10 min, and > 300 m from any DS1 facility geofence.
Route deviation	Vehicle leaves its corridor by 3–15 km for 10–40 min, then rejoins.	Distance to assigned corridor polyline > 2 km for ≥ 3 consecutive pings (broadcast corridor segments, point-to-segment haversine).
Data-quality faults	Malformed JSON, missing keys, out-of-range values, duplicates, late events (up to 30 min), sensor null bursts.	Contract validation → DLQ/quarantine; dedupe; watermark + batch MERGE for late data; null flags preserved.

Independence rule: Member A writes injection, Member B writes detection from the written specification only. The truth log lives in `s3a://truth/`, which the processing jobs cannot read; only the validation harness joins truth to detections.

Volume and run profiles (event time is separate from wall-clock time)

Profile	Vehicles	Ping interval	Simulated span	Events	Use
`smoke`	200	60 s	1 h	~12 k	CI and unit tests
`standard`	5,000	60 s	6 h (compressed ×6 → ~1 h wall)	~1.8 M	Daily integration, dashboards
`full`	5,000	30 s	24 h (compressed ×12 → ~2 h wall)	~14.4 M	Benchmark and final evidence

Full profile ≈ 2,000 events/s at ~350 bytes each (~5 GB raw JSON, ≈ 0.6–0.8 GB as Snappy Parquet). Achieved throughput is measured and reported, not assumed.

4 Engineering design by pillar

Pillar 1 · Domain-specific data generation (owner: A)

- Python 3.11 **asyncio**: vehicles are sharded across tasks (e.g. 50 shards × 100 vehicles), and each shard runs a time-stepped loop. Producers are async (`aiokafka`; `boto3` PutRecords in a thread executor for Kinesis).
- Faker generates driver names, VIN-like IDs and plates; numpy samples the DS2-calibrated distributions.
- `config/generator.yaml` holds seed, profile, anomaly rates and durations, sinks and time compression. The same seed and config give the same output, which satisfies reproducibility.
- **Bulk mode** writes hourly NDJSON micro-batches to `landing/`; an Airflow sensor moves them to bronze. This covers "scheduled bulk micro-batches". A fixed CSV is never used as the only input.

Pillar 2 · Ingestion (owner: A, platform by C)

- **Kafka (KRaft)**: `fleet.telemetry.raw` with 12 partitions keyed by `vehicle_id` (keeps per-vehicle order), `acks=all`, idempotent producer, lz4 compression, 7-day retention. **AWS:** Kinesis Data Streams (2 shards, partition key `vehicle_id`) → Data Firehose → S3 bronze, with the Firehose error-output prefix as the cloud DLQ.

- **Retry with backoff**: exponential backoff with jitter (0.2 s → 6.4 s, 6 tries); after that the record goes to the DLQ topic and a local spill file, so nothing is lost.
- **DLQ**: contract failures from the bronze→silver validation go to `fleet.telemetry.dlq` and `bronze/_dlq/` with `error_code`, `error_detail` and the original payload. `dlq_replay.py` re-submits fixed records, which proves they are recoverable.
- **Ingestion log**: `ops.ingestion_log` (Iceberg) holds batch_id, source, records in/ok/dlq, min/max event time, Kafka offsets and duration per micro-batch.
- **Immutable bronze**: writers only append new `dt/hour` prefixes; bucket versioning is on; downstream jobs use a read-only service account; sealed partitions (older than 2 h) get a MinIO retention lock (`mc retention`) and an S3 deny-delete policy.
- **Batch landing**: a scheduled Python extractor (Airflow) handles the Kaggle CSVs and bulk files: download → checksum → manifest → bronze. NiFi is the documented alternative.

Pillar 3 · Distributed processing (owner: B)

- **Schema enforcement**: explicit `StructType` from the contract; `from_json` with a corrupt-record column; failures go to DLQ.
- **Nulls**: key fields null → quarantine. Sensor nulls are kept and flagged (`reefer_signal_lost`, `gps_gap`), never imputed silently, so disconnections stay visible.
- **Dedupe**: composite business key (`vehicle_id, event_ts, seq_no`); stream uses `withWatermark('event_ts','10 minutes').dropDuplicatesWithinWatermark`; late events past the watermark are reconciled by a batch MERGE.
- **Enrichment**: **broadcast** joins to current vehicle, driver, route and corridor dimensions (all small).
- **Windows**: `lag()` per vehicle for distance and fuel since previous ping; gaps-and-islands for idle, excursion and deviation episodes; rolling 15-min reefer average. No repeated full scans.
- **Shuffle control**: `spark.sql.shuffle.partitions` sized to cores (e.g. 24 locally), AQE on, repartition by `vehicle_id` before windows. DataFrame API only (no `groupByKey`); if RDDs are used, `reduceByKey` / `aggregateByKey`.
- **Storage**: Iceberg tables in Parquet with Snappy; partitioned by `days(event_ts)` and `bucket(16, vehicle_id)`; sorted by vehicle_id, event_ts.

Pillar 4 · Lakehouse & dimensional warehouse (owner: B)

- **Apache Iceberg** for ACID writes, schema enforcement and evolution, MERGE INTO, snapshots and time travel, and hidden partitioning (pruning). Catalog: Iceberg REST locally, AWS Glue Data Catalog in the cloud.
- **Pre-commit quality gate (WAP)**: write to branch `audit_<run_id>`, run the GX suite against that branch, then `CALL system.fast_forward(table,'main','audit_<run_id>')` only if all checks pass. A failed gate leaves `main` untouched and fails the Airflow task.
- **Maintenance**: daily `rewrite_data_files` (small-file compaction), `expire_snapshots` (7-day retention so time travel can be shown), `remove_orphan_files`.

Pillar 5 · Orchestration & analytics (owner: C)

- Apache Airflow on Docker Compose (LocalExecutor). Local Airflow also drives the AWS jobs (EMR Serverless and Athena operators), so there is one control plane. MWAA is optional only if credits allow.
- Superset dashboards on Trino and Athena; every chart is backed by a named SQL view in `gold_kpi`, which makes reconciliation trivial.

5 Dimensional model (gold)

Dimensions

Dimension	SCD	Key attributes (surrogate key `*_sk` on every dimension)
dim_vehicle	**Type 2**	vehicle_id, source_truck_id, make, model_year, reefer_capable, home_depot, reefer_unit_id, status; valid_from, valid_to, is_current

Dimension	SCD	Key attributes (surrogate key `*_sk` on every dimension)
dim_driver	**Type 2**	driver_id, source_driver_id, name (Faker), license_class, home_terminal, assigned_vehicle_id, behaviour_profile
dim_route	**Type 1**	route_id, origin, destination, distance_km, speed_limit_kmh, route_name (corrections overwrite)
dim_facility / dim_customer	Type 1	From DS1; facility lat/lon + geofence radius
dim_cargo	static	cargo_type, is_cold_chain, setpoint_c, tolerance_c
dim_date / dim_time	static	Calendar (day grain) and time-of-day (minute grain)

SCD demo: `fleet.master.changes` reassigns ~2% of drivers and moves ~1% of vehicles between depots mid-run. MERGE closes the old row and opens a new one; facts loaded afterwards point to the new surrogate key, while earlier facts keep the old one. Route name corrections overwrite in place (Type 1).

Facts and grain

Fact	Type	Grain (one row per…)	Main measures
fact_telemetry_ping	Transaction	vehicle reading at event_ts	speed, rpm, temps, fuel_rate, dist_km_since_prev, fuel_l_since_prev, flags
fact_vehicle_hourly	Periodic snapshot	vehicle × hour	km, litres, engine_on_min, idle_min, reefer_min, reefer_in_range_min
fact_trip	Accumulating snapshot	trip (load)	planned/actual km, litres, km/L, departure/arrival ts, idle_min, excursion_min
fact_idle_episode	Transaction (episode)	idle episode	start, end, duration_min, est_fuel_wasted_l
fact_coldchain_excursion	Transaction (episode)	excursion episode	duration_min, peak_temp_c, max_delta_c
fact_route_deviation	Transaction (episode)	deviation episode	duration_min, max_offset_km, extra_km
fact_driver_daily	Periodic snapshot	driver × day	km, harsh_events, speeding_events, idle_min, engine_on_min, safety_score
fact_fuel_purchase	Transaction (DS1)	fuel purchase	litres, cost (historical benchmark)

Metric additivity

Class	Measures	Rule
Additive	km, litres, idle_min, excursion_min, event counts	Sum across any dimension.
Semi-additive	fuel_level_pct, odometer_km, active vehicles	Sum across vehicles at one time; across time use last/avg value, never sum.
Non-additive	km/L, compliance %, idle %, safety score, averages	Never sum or average stored ratios; recompute from additive parts (Σkm / ΣL).

6 KPI definitions (documented, with units and thresholds)

KPI	Definition	Units / assumptions
Fuel efficiency per route	Σ distance_km ÷ Σ fuel_litres per route over the period. Distance = haversine between consecutive pings (gaps > 5 min excluded); fuel = fuel_rate_lph × Δt.	km/L (also L/100 km). Sanity-checked against DS1 fuel purchases.
Cold-chain compliance %	Reefer-active minutes within [setpoint − 2 °C, setpoint + 3 °C] ÷ reefer-active minutes, for cold-chain loads only. Secondary: % cold trips with zero excursions ≥ 15 min.	Percent. Adherence to the *simulation* threshold only, not regulatory certification. Sensor-gap minutes reported separately, not counted as compliant.
Idle %	Idle minutes (episode rule, section 3) ÷ engine-on minutes, per driver per day.	Percent; wasted fuel estimated at idle fuel rate.
Driver safety score	100 − (5 × harsh braking per 100 km + 3 × speeding events per 100 km + 0.5 × idle %), clipped to 0–100. Harsh: max_decel ≥ 3.5 m/s²; speeding: speed > route limit + 10 km/h for ≥ 2 pings.	Score 0–100; weights are documented assumptions, shown on the dashboard.
Pipeline health	Events/min ingested, DLQ rate, freshness lag, GX pass rate, detection precision/recall vs truth.	Operational, for the demo and evidence.

7 Orchestration (Airflow DAGs)

DAG	Schedule	Tasks
`fleet_master_data`	daily / manual	kaggle_download → checksum_manifest → land_bronze → profile → load_dims (SCD MERGE) → gx_dims → publish
`fleet_generator_bulk`	hourly	run_generator_bulk → wait_landing_files → land_bronze → log_ingestion
`fleet_lakehouse_pipeline`	every 15 min	check_new_bronze → silver_telemetry (WAP) → gx_silver → publish_silver → apply_master_changes (SCD) → gold_facts → gold_episodes → kpi_marts → gx_gold → publish_gold → *[cloud branch]* sync_s3 → emr_serverless_gold → athena_checks → superset_cache_refresh → run_report
`fleet_stream_monitor`	5 min	streaming query alive? consumer lag < threshold? DLQ rate < 1%? → alert / restart
`fleet_maintenance`	daily	rewrite_data_files → expire_snapshots → remove_orphan_files → detection_vs_truth_report
`fleet_dlq_replay`	manual	read DLQ → re-validate → resubmit → log outcome

All tasks are idempotent (partition overwrite or MERGE keyed by run window), have `retries=3` with exponential backoff, and log run_id, row counts and durations. Re-running a window must give identical gold results, and that is tested on Day 5.

8 Dashboards (Superset)

1 · Route fuel efficiency

km/L by route (bar), trend by day, worst-10 routes, live vehicle map (deck.gl) coloured by efficiency; filters: date, depot, cargo type.

2 · Cold-chain compliance

Compliance % (big number + trend), excursions by route/customer, temperature timeline for a selected trip with setpoint band, sensor-gap minutes.

3 · Driver safety & idling scorecard

Ranked table (safety score, idle %, harsh/100 km, speeding/100 km), idle hotspots map, wasted-fuel estimate.

4 · Pipeline health & evidence

Events/min, freshness lag, DLQ counts by error code, GX pass rate, detection precision/recall per family, latest run report.

9 Tools and alternatives

Layer	Local (chosen)	AWS (chosen)	Alternatives and when to switch
Generation	Python 3.11, asyncio, Faker, numpy, aiokafka	Same code; boto3 Kinesis sink	threading / multiprocessing; Mimesis; SDV to fit synthetic tables
Data contract	JSON + JSON Schema (versioned in repo)	Same	Avro + Confluent/Apicurio Schema Registry (stronger evolution, +RAM); Protobuf
Streaming broker	Apache Kafka (KRaft) + Kafka UI	Kinesis Data Streams + Data Firehose	Redpanda (lighter, Kafka API: switch if RAM is tight); Amazon MSK / MSK Serverless (higher cost)
Batch landing	Python extractor run by Airflow; MinIO client (`mc`)	`aws s3 sync` / boto3	Apache NiFi; Kafka Connect S3 sink
Raw lake	MinIO (versioning, retention lock)	Amazon S3 (versioning, deny-delete policy)	HDFS; Apache Ozone
Processing	PySpark 3.5 standalone (1 master, 1 worker)	EMR Serverless (Spark 3.5)	AWS Glue ETL (Spark, simpler setup); Databricks; Dataproc
Table format	Apache Iceberg	Apache Iceberg	Delta Lake (great on Databricks; weaker Athena/Trino write story); Apache Hudi
Catalog	Iceberg REST catalog (+ Postgres)	AWS Glue Data Catalog	Project Nessie (git-like branches); Hive Metastore; Apache Polaris
SQL engine	Trino	Amazon Athena (engine v3)	Presto; Hive on Tez; Spark Thrift Server; ClickHouse; Redshift Spectrum / Snowflake
Data quality	Great Expectations (GX Core)	Same, run from Airflow	Soda Core; PyDeequ; custom Spark assertions
Orchestration	Apache Airflow (Docker Compose, LocalExecutor)	Same local Airflow driving AWS operators	Amazon MWAA (managed, paid); Dagster; Prefect; Step Functions
BI	Apache Superset	Superset → Athena (PyAthena)	Metabase; Power BI Desktop via Athena ODBC; Amazon QuickSight; Grafana
Infrastructure	Docker Compose, Makefile, `.env` profiles	Terraform (S3, Kinesis, Firehose, Glue, EMR Serverless, Athena, IAM, Budgets)	AWS CDK; CloudFormation; plain AWS CLI scripts
Monitoring	Kafka UI, Spark UI, Airflow logs, MinIO console	CloudWatch	Prometheus + Grafana

Layer	Local (chosen)	AWS (chosen)	Alternatives and when to switch
Engineering	GitHub (PRs + review), pre-commit (ruff, black, sqlfluff), pytest + chispa for Spark tests, GitHub Actions CI (lint + smoke tests)		GitLab CI
Docs & diagrams	Markdown README, Mermaid/draw.io architecture, dbdiagram.io ERD, GX Data Docs (HTML)		Excalidraw; MkDocs site

Local resource budget (16 GB laptops)

Give Docker / WSL2 about 11–12 GB (`.wslconfig memory=12GB` on Windows). Compose profiles let each member run only what they need.

Profile	Services	Approx. RAM
`core`	Kafka 1 GB, MinIO 0.5 GB, Iceberg REST + Postgres 0.7 GB, Spark master 0.5 GB + worker 4 GB	~6.7 GB
`orchestrate`	Airflow scheduler + API/webserver + triggerer (LocalExecutor, shared Postgres)	~2 GB
`serve`	Trino 2 GB, Superset 1 GB	~3 GB

One laptop is the designated demo machine for the full stack. If memory pressure appears, switch Kafka to Redpanda and run Superset only against Trino.

10 Team roles and ownership

Member A · Data & Streaming

Pillars 1–2. Kaggle profiling and data contract; DS2 calibration; corridor builder; 5,000-vehicle generator, anomalies, DQ faults, truth log; Kafka + Kinesis producers; retry/backoff, DLQ, replay tool; ingestion log; detection-vs-truth harness; throughput benchmarks.

Member B · Processing & Lakehouse

Pillars 3–4. Spark env profiles; bronze stream job; silver cleaning, dedupe, enrichment, windows; star schema and SCD1/SCD2 MERGE; episode detection (independent of A); KPI marts; GX suites + WAP; Iceberg evolution, time travel, pruning, compaction; tuning.

Member C · Platform, Cloud & Analytics

Pillar 5 + infra. Repo, CI, Docker Compose; Terraform for AWS and budget guardrails; Airflow DAGs for both pathways; Trino and Athena; Superset dashboards; KPI reconciliation; README, evidence pack, demo recording; deck lead.

Responsibility matrix (R = responsible, A = accountable/approves, C = consulted)

Deliverable	A	B	C
Data profile, contract, calibration, generator	R/A	C	C
Kafka / Kinesis ingestion, DLQ, ingestion log	R/A	C	R (infra)
Silver jobs, dedupe, enrichment	C	R/A	
Dimensional model, SCD, gold facts, KPI marts	C	R/A	C
Quality gates (GX + WAP)	R (bronze/contract)	R/A (silver/gold)	
Docker platform, AWS Terraform, cost control		C	R/A
Airflow DAGs	C	C	R/A
Dashboards + KPI reconciliation		R (SQL)	R/A
Testing, evidence pack	R	R	A
README, architecture diagram, deck, demo video	R (own parts)	R (own parts)	R/A

Working agreement

- **Branching:** `main` protected; feature branches; every PR reviewed by one teammate (rotation A→B, B→C, C→A) against the review checklist (section 13).
- **Rhythm:** 15-min stand-up at start of day; **integration point at 17:00** daily, when everything merged so far runs end-to-end on the demo machine; end-of-day gate review.
- **Interfaces first:** the data contract (Day 1) and table DDLs (Day 1) are frozen early, so A, B and C can build in parallel against them.

11 Six-day execution plan (mapped to the handbook's readiness gates)

Day	Member A	Member B	Member C	Exit gate (must be true to close the day)
Day 1 Scoping & foundations Gate 1	Download + profile DS1/DS2 (keys, nulls, units, ranges); data contract v1; city lookup + corridor builder; generator skeleton (50 vehicles)	Bus matrix, ERD v1, grain statements, SCD decisions; KPI formula sheet (with C); Spark session factory with env profiles; Iceberg DDLs	Repo, structure, pre-commit, CI; Docker Compose with profiles and memory limits; AWS: budget alarm, IAM, S3 buckets, Glue DBs via Terraform	Scope statement, objectives and ownership agreed; architecture v1; contract + DDLs reviewed; compose healthy; one hand-made event goes Kafka → MinIO
Day 2 Thin slice end-to-end Gate 2	Generator at 5,000 vehicles on corridors, trips from DS1 loads, asyncio shards, seeded config; stream + bulk modes; retry/backoff + DLQ; Kinesis sink	Bronze streaming job (Kafka → MinIO raw + metadata); DS1 master load → silver dims (initial); silver telemetry v1 (parse, cast, dedupe)	`fleet_master_data` DAG; Trino + Superset connected to Iceberg; Kinesis → Firehose → S3; bronze mirror; Athena query on bronze	A generated event is traceable Kafka → bronze → silver → Trino **and** Kinesis → S3 → Athena; architecture v2
Day 3 Core build I Gate 3 (part)	3 anomaly families + DQ faults + sealed truth log; master-change events; `ops.ingestion_log`; DLQ replay; throughput benchmark v1	Silver complete (null policy, quarantine, broadcast enrichment, corridor distance, lag windows, watermark); SCD2 vehicle/driver, SCD1 route MERGE; fact_telemetry_ping, fact_vehicle_hourly, fact_trip	`fleet_lakehouse_pipeline` skeleton (task groups, spark-submit + EMR operators); KPI SQL v1; dashboard wireframes; Terraform EMR Serverless app + Athena workgroup	5,000 vehicles flowing continuously; SCD history visible in dims; first facts queryable in Trino
Day 4 Core build II Gate 3	Detection-vs-truth harness (precision, recall, detection delay); GX bronze/contract suite; DLQ evidence; schema v2 (`tire_pressure_kpa`)	Episode facts (idle, excursion, deviation) via gaps-and-islands; fact_driver_daily + safety score; KPI marts; GX silver/gold + WAP; maintenance jobs	4 Superset dashboards; Athena datasource; full DAG run locally and the cloud branch (EMR Serverless → Glue → Athena); run report	One orchestrated run goes raw → dashboard in both environments; all 3 KPIs visible
Day 5 Testing & refinement Gate 4	Failure tests: broker restart, malformed burst, Kinesis throttling → retry, late events; full-profile benchmark	KPI reconciliation (dashboard = SQL = hand calc); tuning (EXPLAIN pruning, broadcast, shuffle partitions, compaction); time travel + rollback demo	DAG failure/retry and idempotent re-run tests; README run guide; fresh-clone reproduction on 2nd laptop; AWS cost check	Every acceptance criterion demonstrable with saved evidence; defects fixed or logged as limitations
Day 6 Hardening & demo package Gate 5 prep	Data assets folder (schemas, configs, sample events, anomaly docs); deck: data & ingestion	Lakehouse evidence (table history, SCD, time travel, GX docs); model docs; deck: processing & model	Final architecture diagram; recorded fallback demo; deck assembly; contribution summary; AWS teardown script	**Feature freeze 12:00.** Final `full` run; two demo rehearsals; readiness checklist fully ticked

> **Critical path:** data contract (D1) → generator at scale (D2) → silver (D3) → episodes + KPIs (D4) → reconciliation (D5). Anything not on this path can slip; anything on it is swarmed by the whole team at the 17:00 integration point if late.

12 Testing and validation plan

ID	Test	Pass criteria	Evidence / owner
T01	Generator coverage	5,000 distinct vehicles; all fields; all 3 anomaly families + DQ faults present	Count queries, truth-log summary · A
T02	Reproducibility of generator	Same seed + config → identical event hash for smoke profile	pytest · A
T03	Malformed burst	Every bad record in DLQ with error_code; replay restores fixed ones	DLQ logs, replay report · A
T04	Broker outage (stop Kafka 2 min)	Producer retries with backoff; no loss after restart (counts reconcile)	Producer log, reconciliation · A
T05	Duplicates and late events	0 duplicates in silver; late events merged into the correct partition	GX report, MERGE history · B
T06	Count reconciliation	bronze = silver + quarantine + DLQ per run	Reconciliation view · B
T07	Quality gate blocks bad data	Injected bad batch fails GX; `main` snapshot unchanged; Airflow task red	Iceberg refs, Airflow log · B
T08	SCD behaviour	Reassigned driver has 2 rows; old facts keep old SK; Type 1 route overwritten	Before/after queries · B
T09	Time travel + rollback	Query previous snapshot by version and timestamp; rollback restores counts	Trino + Athena queries · B
T10	Partition pruning	One-day filter reads ≤ 1/N of files	EXPLAIN ANALYZE output · B
T11	Detection accuracy	Recall ≥ 95%, precision ≥ 90% per family	Harness report · A
T12	KPI reconciliation	Dashboard = SQL = hand calc on a sampled vehicle/route/driver (±0.1%)	Reconciliation sheet · C
T13	Idempotent re-run	Re-running a DAG window yields identical gold hashes	Airflow logs, hash query · C
T14	Cloud parity	Same window in Trino and Athena gives same KPI values	Side-by-side queries · C
T15	Fresh-clone reproduction	Second laptop runs `make up && make demo` in ≤ 30 min	Screen recording · C
T16	Performance (measured)	Events/s, end-to-end latency, job runtimes, RAM/CPU recorded	benchmark.csv, Spark UI · A + B

13 Repository, conventions and senior review

```
fleet-lakehouse/
├─ Makefile  .env.example  README.md
├─ docker/            compose.yaml, service configs
├─ infra/terraform/   aws: s3, kinesis, firehose, glue,
│                     emr-serverless, athena, iam, budget
├─ conf/              local.yaml  aws.yaml  units.yaml
├─ contracts/         telemetry.v1.json  v2  master_change
├─ generator/         fleet_sim/ (async shards, corridors,
│                     anomalies, sinks)  generator.yaml
├─ ingestion/         extractors, dlq_replay.py
├─ spark_jobs/        bronze/ silver/ gold/ maintenance/
├─ quality/           gx/ suites + checkpoints
├─ sql/               ddl/  kpi_views/  reconciliation/
├─ airflow/dags/      fleet_*.py
├─ dashboards/        superset exports (zip/yaml)
├─ tests/             unit/ spark/ e2e/
├─ data_samples/      sample events, profiles
└─ evidence/          gx_docs, logs, screenshots, bench
```

Definition of done (every PR)

- Runs with `ENV=local`; cloud-sensitive code also runs with `ENV=aws`.
- Idempotent; no hard-coded paths or secrets (use `.env` / IAM roles).
- Unit or Spark test added; smoke profile passes in CI.
- Schema or contract changes are versioned and documented.
- Row counts and timings are logged; failures are visible, never swallowed.
- KPI changes update the definition table and reconciliation SQL.
- Evidence (screenshot/log) saved to `evidence/` when it proves a requirement.

Senior review checkpoints

- End of D1: contract, ERD, grain, KPI formulas (no build before sign-off).
- End of D2: thin slice in both environments.
- End of D4: full run + dashboards.
- End of D5: test plan results, evidence pack.

14 Risks, mitigations and AWS cost guardrails

Risk	Impact	Mitigation
Kaggle schemas differ from expectations	High	Day 1 profiling; adapter layer at silver boundary; contract frozen only after profiling.
16 GB RAM pressure with full stack	High	Compose profiles, memory limits, one demo machine, Redpanda fallback, smaller run profile for development.
Iceberg catalog differences (REST vs Glue)	Med	Catalog set only in env config; parity test T14 from Day 2, not Day 5.
WAP/branch features unsupported in a tool version	Med	Fallback: staging table + GX + atomic MERGE into main (same guarantees).
Detection leaks labels	Med	Truth log in separate bucket with no read grant for jobs; A/B independence rule.
Streaming instability during demo	Med	Recorded fallback video; pre-loaded `full` run so dashboards are populated before going live.
Unexpected AWS charges	Med	Budget alarms at $5/$10/$20; Kinesis and EMR only during scheduled test windows; `make aws-down` teardown; S3 lifecycle on athena-results.
Integration surprises at the end	High	Daily 17:00 end-to-end integration from Day 2 onward.

Cost note: with streams and EMR switched on only during test windows, the week should stay in the low tens of dollars at most (S3, Kinesis shard-hours, Firehose GB, EMR Serverless vCPU-hours, Athena per TB scanned). Confirm with the AWS Pricing Calculator on Day 1 and set the budget alarm before creating any streaming resource.

15 Submission package mapping

Handbook deliverable	What we hand in	Owner
01 Source code	Repository above: generator, ingestion, Spark, warehouse SQL, Airflow DAGs, Terraform, compose	All

Handbook deliverable	What we hand in	Owner
02 Technical documentation	README (setup for local and AWS, dependencies, configuration, run steps), architecture diagram, ERD, KPI definitions	C
03 Data assets	Generator, calibration, contracts v1/v2, anomaly specs, sample events (incl. DLQ samples), Kaggle profiling report	A
04 Technical evidence	GX Data Docs, ingestion and DLQ logs, Spark outputs, Iceberg history/snapshots, SCD before/after, Airflow logs, detection report, benchmarks	A + B
05 Working solution	Executable pipeline in both environments, Iceberg lakehouse, star schema, KPI views, Superset dashboards	All
06 Presentation deck	Problem, architecture, implementation, results, limitations, business value	C (lead)
07 Demonstration	Live demo script (≤ 12 min) + recorded fallback	C
08 Contribution summary	Per member: components built, evidence produced, PRs reviewed	All

Requirement traceability (Project 03 acceptance criteria)

Acceptance criterion	Where it is satisfied	Test
Configuration and output evidence cover 5,000 distinct vehicles	generator.yaml; dim_vehicle; silver distinct count	T01
All telemetry fields and three anomaly families present	Contract v1; truth log; episode facts	T01, T11
Route, cold-chain and driver KPIs reconcile with analytical queries	gold_kpi views; reconciliation SQL; dashboards	T12, T14
A complete orchestrated run connects raw events to warehouse and dashboard	`fleet_lakehouse_pipeline` + run report	T13, T15
Shared: immutable raw, DLQ, retry, logs	Pillar 2 design	T03, T04
Shared: ACID, time travel, MERGE, pruning, SCD1/2, star schema, quality gates	Pillar 4 design + section 5	T07–T10
Recommended: units/thresholds stated; throughput and resources recorded	KPI table (section 6); benchmark.csv	T16

16 Open items to confirm

- Member names, to replace A / B / C in the deck and contribution summary.
- Exact DS1/DS2 schemas (Day 1 profiling): file names, units, ID formats.
- Whether the AWS account is a personal account with credits or a lab account (some lab accounts block EMR Serverless; then switch to Glue ETL).
- Demo date and slot length, so the demo script and recording are sized correctly.